

为什么 Attractor Guided Engineering 不能被降级为 AI Agent Skill

原创 可逆计算 可逆计算 2026年6月1日 14:48 北京

Agent Skill 已经是 AI Agent 工程中最容易被接受的实践之一：把可重复任务封装成可发现、可调用、可注入上下文的能力包。

这当然有价值。诊断 bug 可以有 skill，审查代码可以有 skill，生成 PPT 可以有 skill，转换文件可以有 skill。skill 让 Agent 更快进入一种经过验证的工作方式，也让团队把经验沉淀成可复用的工作说明书。

但正如那句俗语所说：“如果你手里只有一把锤子，所有东西看起来都像钉子。” Skill 也是如此：一发现某个有用实践，很多人的第一反应就是“把它包装成一个可复用 skill”。

吸引子引导工程理论 (Attractor Guided Engineering, AGE) 指出，如果把所有 AI 工程实践都 skill 化，就会漏掉 AI 主导软件工程中最关键的问题：**系统如何在持续扰动下仍然收敛？**

Skill 解决的是能力调用问题。AGE 解决的是领域结构在仓库长期演化中如何被保持，并如何抑制轨迹漂移的问题。

这也解释了为什么 AGE 不能被降级为一个 skill：把 skill 作为 AGE 实践的主载体，会把仓库级收敛机制降成一次性调用能力，无法实现 AGE 的目标。

一、Skill 的本质是语义级 hash map

很多人理解 skill 时，会先看到它的文件形式：`SKILL.md`、description、trigger words、supporting files、scripts、templates。

但这些只是外壳。skill 真正起作用的地方，是 Agent runtime 会把它们组织成一个**语义级的查找表**：

```
task intent -> matched skill -> procedure bundle
```

也就是说，Agent 并不是因为某个文件叫 `SKILL.md` 才获得能力，而是因为当前任务意图可以在 skill 描述空间中匹配到一个可加载的工作说明书。

这里的 hash 不是字符串 hash。key 不是固定字符串，而是任务意图、上下文线索、触发词和描述的组合；value 也不是单个文件，而是一组可以被注入上下文的说明、脚本、模板和示例。

这种结构非常适合回答一个问题：

我现在要做这类事，应该加载哪套做法？

所以 skill 很适合封装局部能力：诊断 bug、审查 diff、调用工具、生成报告、执行 checklist。它的组织轴是调用意图，把“要做什么”映射到“加载哪个能力包”。

二、信息形式不是关键

既然 skill 真正起作用的是语义匹配，那么就不能反过来迷信 skill 这种文件形式。

对智能体来说，信息写成什么格式并不是本质。AI 可以读 Markdown，也可以读 XML 等其他格式。数据库模型、API 模型等完全可以用 DSL（领域特定语言）来表达。

`AGENTS.md` 和 `docs/index.md` 完全可以承担信息路由，甚至可以做得更好：通过文件链接，我们可以按领域结构提供分层路由。AI 可以先读紧凑入口，再按任务需要逐层打开更具体的 owner doc、模型文件、测试和源码。

因此，问题不在于“这条知识有没有被写成 skill”。

问题是：**信息换了形式之后，原来的结构还在不在。**

智能的一个关键能力就是可以自由地跨形式转换信息。如果结构还在，知识写成 Markdown、XML、测试、DSL/schema、源码锚点都可以被 AI 使用。如果结构丢了，把它包装成再精致的 skill，也只是让丢失结构后的片段更好调用。

三、为什么动力系统关心保结构

数学物理中的动力系统提供了一个很好的直觉：描述系统的变量可以改变，但支配系统演化的关键结构不能随便丢。

想象一个摆钟。你可以用角度和角速度描述它，也可以用位置和速度描述它，还可以用能量和相位描述它。表达形式可以变，但摆钟的关键结构不能丢：位置和速度怎样耦合，能量怎样变化，阻尼怎样消耗能量，外力从哪里进入。

如果一个算法只追求“下一步位置看起来差不多”，却不保持这些结构，它可能每一步误差都不大，长期却让摆钟无缘无故越摆越高，或者突然停下来。局部输出像，长期系统不再是原来的系统。

再看一个电路。你可以用节点电压描述，也可以用支路电流描述，还可以把电容、电感里的能量作为状态变量。变量形式可以变，但有些结构不能丢：电荷守恒、电压电流的耦合关系、元件储能和耗散、外部电源从哪里注入能量。

如果一个算法只追求某几个采样点的电压值看起来接近，却不保持这些结构，长期仿真可能会凭空产生能量，或者把本来应该耗散的系统算成振荡不止。局部曲线像，系统物理意义已经错了。

这就是保结构：

表达形式可以变，关键约束、耦合关系和守恒/耗散结构不能丢。

一个受控系统要长期收敛，不是靠每一步看起来合理，而是靠这些结构在反复变换和扰动中被保持。

四、状态空间不是源码空间

如果借助于动力系统的视角来观察软件工程，仓库就不再是一个静止的文件夹，而是一个反复被扰动的系统。每一次需求澄清、计划生成、代码修改、测试补充、审查和日志更新，都会把仓库推到一个新状态。

传统软件工程里存在一个底层的隐含假定：人脑可以长期充当系统真相的隐式整合器。源码、需求、架构取舍、历史 bug、未写下来的上下文，最终都可以由少数核心工程师完整理解并决定演化方向。

AI 深度参与之后，这个底层的假定被动摇了。AI 可以高频生成、高幅修改、跨模块扩散，而每个 session 又缺少持续方向感。AI 在状态空间中引入的扩张力远远超过了少数人类的信息处理能力。此时，除非强行把 AI 降速到人类可以持续整合的节奏，否则系统演化过程中的真相源就不可能再是人的脑内记忆。下一个 session 能重新读取的，不是作者脑中的完整意图，而是仓库里的代码、diff、日志、测试、文档、模型和审计证据。

`nop-chaos-flux` 的历史记录里已经多次出现这种漂移。[docs/plans/436-deep-audit-2026-05-24-full-remediation-plan.md](#) 的 Workstream 8 修复过 active-doc drift：文档把 `component:open/close/toggle`、`component:refresh`、旧 package split、旧源码锚点和过期 component inventory 描述成当前事实，但 live repo 已经不是这样。[docs/logs/2026/05-26.md](#) 又记录过 proof drift：E2E 仍断言 spreadsheet 的 30 个 row headers 同时挂载，而 live spreadsheet shell 已经虚拟化，正确证明只能断言可见行头挂载。代码、文档、测试各自都可能局部合理，但它们之间的 owner/proof/precedence 关系已经漂移。

因此，AGE 的基本前提是：

仓库 = 真相源 (Source of Truth)
Chat = 临时工作面

在这个前提下，核心问题不是：

这次 Agent 有没有调用对 skill？

而是：

仓库经过很多次 Agent 扰动后，是否仍然沿着领域结构受控收敛？

AGE 的基本图像是：

状态空间 -> 吸引子 -> 轨迹 -> 控制

这里的状态空间不是“源码状态空间”。源码回答当前实现是什么，但它不能单独回答系统应向哪里收敛、某个设计为什么成立、哪条行为已经被证明、某次偏离为什么被接受、下一次 AI 应该从哪里恢复上下文。

AGE 关心的是整个仓库工程现实的状态：源码、测试、owner docs、requirements、plans、logs、bug notes、schemas、XML 模型、数据库模型、XDSL、[AGENTS.md](#)、[doc s/index.md](#)、CI 配置和审计证据共同组成一个状态。

这些东西不是几套可分离材料，而是同一组语义承诺在不同载体上的分布。代码对实现事实有权威，owner doc 对收敛方向有权威，测试对已证明行为有权威，logs / bug notes 对演化轨迹有权威，plans / audits 对本轮变化是否闭合有权威。AGE 要维护的是这些权威之间的关系，而不是把某一种文件格式神圣化。

Skill 的位置不同。**AGE 要组织的是仓库自身；skill 是作用于仓库的外部能力或控制输入。**

外部能力可以改变系统，但不能替代系统内部结构。一个 [bug-diagnosis](#) skill 可以用于很多仓库、很多领域、很多 bug 类型。它的组织逻辑是通用任务能力，不是某一个仓库内部的领域概念拓扑。

仓库内部真正需要保持的是源码、文档、测试、模型、计划、日志、审计证据之间的 owner、proof、precedence、freshness 等关系。Skill 可以帮助修改这些东西，却不能替代这些关系本身。

这时会发现，skill 的语义 hash 不足以解决问题。Skill 保存的是：

任务意图 -> 能力包

AGE 要保持的是：

领域概念 -> 语义承诺 -> 实现位置 -> 证明证据 -> 审计/记忆 -> 后续义务

这两种结构不是一回事。Skill 可以告诉 Agent 如何 review，但不能自动知道哪个 owner doc 拥有当前语义事实。Skill 可以告诉 Agent 如何写测试，但不能自动知道这个测试保护哪条领域承诺。Skill 可以告诉 Agent 如何更新文档，但不能自动判断某条信息应该进入 owner doc、bug note、log、lesson、reference 还是 skill。

这些判断不是调用能力本身，而是仓库的语义权威结构。

五、为什么 age-skill 是错误抽象

把 AGE 做成一个 `age-skill`，表面上很诱人：把 AGE 的规则、流程、检查清单、文档模板都写进一个可调用能力包。Agent 需要 AGE 时，就加载这个 skill。

但这正好把 AGE 放错了层级。Skill 是任务发生后被匹配和加载的能力包；AGE 应该是任务开始前就已经存在的仓库结构。它决定任务如何被理解、信息从哪里读、冲突听谁的、完成后如何证明闭合。

AGE 不能等到某个 skill 被选中之后才出现。它应该已经体现在：

- `AGENTS.md` 的操作边界
- `docs/index.md` 的路由结构
- owner docs 的事实归属
- source-of-truth precedence 的冲突规则
- plans 的义务声明
- tests 的 proof relation
- audits 的 closure gate
- logs、bug notes、lessons 的轨迹记忆
- freshness/autonomy 的行动限制

如果这些关系只存在于 `age-skill` 里，它们就不是仓库结构，而只是一个 procedure bundle 的内容。只要 Agent 没有匹配到这个 skill，或者 skill 与 live repo 冲突，AGE 就失效了。

把 `age-skill` 做成 always-on 或 global skill 也不能解决这个问题。它最多能规定“先读哪些 owner docs、如何检查 precedence、怎样做 closure audit”。但真正的 owner、proof、precedence、freshness 关系仍然必须存在于仓库文件、测试、日志、计划和审计证据之间。

所以问题不在于 `age-skill` 是不是每次都会被加载，而在于 skill 仍然是从外部施加到仓库上的能力包；AGE 要成为仓库自身的底层拓扑。只要 owner、proof、precedence、freshness 没有内化为仓库状态，AGE 就被降级成了外部操作方法，而不是内在收敛结构。

六、AGE 补上的缺失概念

在受控收敛和保结构的图像下，AGE 补上的不是一个新 checklist，而是一组 Skill 化实践缺少的一等系统概念（First Class Concept）。

1. 吸引子

系统长期应该回到什么结构？这不能由某个 skill 决定。它需要 owner docs、架构基线、领域设计和 source-of-truth precedence 来承载。没有吸引子，skill 只能告诉 Agent 怎么做事，不能告诉仓库应该向哪里收敛。

2. 轨迹

单次任务完成不等于系统方向正确。logs、bug notes、lessons、plans、audit records 记录的是仓库如何一步步走到现在。没有轨迹记忆，AI 每次都像从局部截面重新开始，很难判断系统是在收敛还是漂移。

3. 语义权威

当 skill、plan、代码、测试、文档说法不一致时，听谁的？AGE 需要 owner、routing、precedence，而不是让当前被调用的 skill 临时裁决。语义权威不是执行步骤的一部分，而是仓库状态空间内部的秩序。

4. Proof relation

测试不是“有覆盖”就够了。测试要说明保护哪条语义承诺。audit 也不是 checklist，而是检查承诺是否消失、弱化或转移到非权威载体。proof relation 让验证不只是通过命令，而是回到领域承诺。

5. Freshness / autonomy

文档不是写了就永远可信。文档是否 fresh，会影响 AI 能不能基于它行动，能自动走多远，什么时候必须停下来问人。freshness / autonomy 把“能不能信、能不能自动做”变成仓库可见状态，而不是一次会话里的临时判断。

6. 保结构的文档路由

[AGENTS.md](#) 可以给 AI 一个紧凑的操作入口，[docs/index.md](#) 可以提供完整路由。二者不是为了把所有知识塞进同一个文件，而是为了让 AI 按任务需要逐层打开正确的信息。信息披露可以分层，但领域结构不能被打散。

这些概念加在一起，才构成 AGE 的语义权威拓扑。

七、AGE 可以使用 skill，但不能变成 skill

AGE 不反对 skill。skill 是很好的执行支架。

但 skill 在 AGE 中只能是方法节点，不能是真理源，也不能是 AGE 本身。

正确顺序是：

```
先按 AGENTS.md / docs/index.md 找路由
再读领域或架构 owner doc
再读 active requirement / plan / audit evidence
最后选择合适的 skill 作为执行方法
```

不是：

```
先选 skill，再让 skill 决定事实归属
```

例如 bug diagnosis skill 可以告诉 Agent 如何复现和定位问题。但根因属于哪个领域概念、哪个 owner doc 要更新、哪个测试证明修复、bug note 以后约束哪类回归，这些必须回到 AGE 的语义拓扑中决定。

好的 skill 加速执行。坏的 skill 化，用执行方法替代领域结构。

八、判断标准

判断一套 AI 工程实践是不是过度 skill 化，可以问：

1. 它主要按操作动词组织知识，还是按领域概念和架构 owner 组织知识？
2. 它的链接主要是执行资源引用，还是 owner / invariant / proof / precedence / freshness 关系？
3. 信息在 skill、plan、code、test、doc、log 之间转换后，领域结构是否仍可恢复？
4. 删除所有 skills 后，仓库是否仍知道什么是对的、谁拥有它、如何证明它？

如果答案是否定的，skill 已经承担了不该承担的吸引子职责。

最终区别可以压缩成一句话：

```
Skill packages capabilities by invocation.
AGE preserves domain structure across transformations.
```

Skill 让 Agent 更会做事。AGE 让仓库在 Agent 反复做事之后，仍然沿着领域结构受控收敛。

AGE 应用开发模板

- GitHub: <https://github.com/entropy-cloud/attractor-guided-engineering-template>
- Gitee: <https://gitee.com/canonical-entropy/attractor-guided-engineering-template>

稀罕作者

可逆计算 · 目录

上一篇

Harness Engineering 之外：从非线性动力系
统控制理解吸引子引导工程

下一篇

自进化的两个尺度：RMSP Agent 与 AGE 方
法论的深层结构对应